



OPEN Real time task planning for order picking in intelligent logistics warehousing

Shaohui Zhang^{1,2}✉, Qiuying Han³, Hai Zhu⁴✉, Hongfeng Wang¹, Huiling Li² & Ke Wang²

With the rapid growth of e-commerce and ongoing innovations in the logistics industry, intelligent unmanned logistics warehousing systems have emerged to significantly enhance operational efficiency and reduce costs. In these systems, the two critical stages of order assignment and path planning are interconnected through racks in the order picking process. However, prior research has largely overlooked their joint optimization. In this paper, we investigate the real time task planning problem (RTTP) in intelligent unmanned logistics warehousing, where racks are dynamically assigned to orders arriving in real time, and robots are responsible for delivering the racks to workstations according to planned paths, with the goal of jointly minimizing total order processing time and travel costs. To solve the RTTP, we first design a joint optimization evaluation indicator and propose a joint optimization task planning (JOTP) algorithm. Furthermore, we innovatively introduce a reinforcement learning-based approach (JOTP-RL) by modeling order selection as a partially observable Markov decision process (POMDP), and employing the Q-Mix algorithm to solve it. To enhance path planning efficiency, we optimize the improved THA* algorithm by eliminating redundant calculations and accounting for congestion times. Finally, extensive experiments conducted on two datasets demonstrate that our proposed algorithms significantly outperform the baseline and state-of-the-art methods, achieving superior efficiency and effectiveness in both execution time and task optimization.

The boom of e-commerce has led to a surge in logistics demand. This explosive expansion, combined with advancements in smart warehousing and the integration of Industry 4.0 technologies—such as IoT (Internet of Things)^{1,2}, RFID (Radio Frequency Identification)^{3,4}, and AI (Artificial Intelligence)^{5,6}—has significantly increased the demand for logistics services. Smart warehousing technologies facilitate more efficient and flexible warehouse management by automating inventory tracking, order fulfillment, and internal transportation. As a crucial component of urban logistics, order picking in logistics warehouses has a significant impact on the overall efficiency of the distribution process. This process includes everything from order receipt to item retrieval, and ultimately the successful packaging of all involved items. Traditional logistics warehouses employ the “picker-to-item” paradigm for order picking, wherein pickers follow the aisle or take vehicles to fulfill single or batch orders of multiple items. However, when meeting large-scale orders and items, it can be very time-consuming and labour-intensive^{7–9}. With the development of automation technology, the Robot Mobile Fulfillment System (RMFS) has emerged, which leverages various types of robots to relocate racks across areas¹⁰, thereby promoting the success of intelligent unmanned logistics warehousing. Notable examples of this technology in use include companies like Amazon, Cainiao, and JD, which have adopted automated and robotic solutions to streamline warehouse operations and improve efficiency.

Order picking is one of the key challenges in logistics warehouses, involving both order assignment and path planning¹¹. Order assignment refers to the process of selecting appropriate racks for each order to ensure all items are covered, while path planning involves determining the optimal paths for robots to transport racks, enabling their swift relocation to workstations. Together, these two stages are collectively referred to as task planning^{11,12}. Regarding order assignment, in the single workstation scenario, Yang et al.¹³ proposed an optimization algorithm for order-picking and rack-moving based on a mixed integer linear programming model to minimize the number of rack movements. Zhuang et al.¹⁴ proposed an adaptive large-neighborhood search algorithm and simulated annealing method for multiple workstation scenarios. Shi et al.¹¹ proposed a two-stage hybrid optimization of order picking and rack assignment based on dynamic programming and neighborhood search to minimize the number of visits to workstations. As for path planning, Shi et al.¹² designed an efficient

¹School of Artificial Intelligence, Zhoukou Normal University, Zhoukou 466001, China. ²School of Computer and Artificial Intelligence, Zhengzhou University, Zhengzhou 450001, China. ³School of Computer Science and Technology, Zhoukou Normal University, Zhoukou 466001, China. ⁴School of Computer, Henan University of Engineering, Zhengzhou 451191, China. ✉email: zhangsh@zknun.edu.cn; zhu_sea@163.com

adaptive task planning framework to solve the problem of moving paths and sequences of large-scale warehouse robots in the high throughput scenario. Others, such as the CBS algorithm¹⁵, ECBS algorithm¹⁶, and eMDD-SAT algorithm¹⁷, can plan conflict-free paths in a grid environment for multi-robots.

In summary, there has been extensive study on order assignment^{18–21} and path planning^{22–25} as separate sub-problems, with less attention given to the joint optimization of order assignment and path planning, i.e., task planning. However, the two stages of order assignment and path planning are interconnected through racks in the order picking, and their joint optimization exhibits mutual influence and dependence. For example, a reasonable assignment can reduce the cost of rack movement, thereby improving the overall efficiency. Meanwhile, optimized path planning can ensure that robots relocate racks quickly, reducing time consumption. In this paper, our contributions are summarized as follows:

- The Real-Time Task Planning (RTTP) problem in RMFS is formulated with the objective of minimizing total order completion time and travel costs, while addressing key aspects of real-time task management.
- A joint optimization evaluation indicator is introduced to assess the contribution of each rack to the order fulfillment process, aiming to optimize overall order processing time by considering both rack content and location factors. Building on this, a batch-based Joint Optimization Task Planning (JOTP) algorithm is proposed to allocate orders to the most suitable racks within each batch.
- Furthermore, the RTTP problem is modeled as a partially observable Markov decision process (POMDP), and a multi-agent joint optimization task planning algorithm based on reinforcement learning (JOTP-RL) is proposed, which is solved using the centralized training with decentralized execution (CTDE) framework. To enhance path planning efficiency, we optimize the improved THA* algorithm by eliminating redundant calculations and accounting for congestion times.
- Extensive experiments have been conducted to thoroughly evaluate the efficiency and effectiveness of the proposed algorithm. The experimental results demonstrate that our algorithm outperforms the baseline and state-of-the-art methods, delivering superior performance in both execution time and task optimization. The rest of this paper is organized as follows: In “Related works” section, we review relevant studies and research in the field. The “Problem definition” section formally defines the RTTP problem. In the “Solution” section, we present the proposed solutions, including the JOTP and JOTP-RL methods. The “Experiments” section discusses the experimental results. Finally, the paper concludes with a summary and outlines future research directions in the “Conclusion and future works” section.

Related works

Our study is related to two threads of studies. In this section, we review a set of related works that focus on the task assignment and path planning in warehouses.

Group task assignment

In order picking, multiple racks are required to cooperate to complete several orders, which can be regarded as the group task assignment problem¹¹. As such, we first review existing works on group task assignment. Existing study on group task assignment can be broadly categorized into heuristic methods, prediction-based methods, data-driven methods, and reinforcement learning-based methods.

Heuristic methods are widely employed to tackle group task assignment across various scenarios, as they offer feasible solutions while meeting fundamental efficiency requirements. For example, Li et al.²⁶ studied the utility-aware dynamic task assignment in crowdsourcing, aiming to maximize the utility of task matching, and designed several heuristic algorithms from efficiency and matching quality perspectives. Regarding prediction-based methods, Tong et al.²⁷ focused on flexible online task allocation in real-time spatial data and proposed a prediction-based online task assignment algorithm with a competition ratio of 0.47. Analyzing and leveraging large-scale historical data is another critical approach to improving the solution quality of group task assignment. As for data-driven methods, Zhao et al.²⁸ proposed a two-stage data-driven task assignment framework comprising a prediction phase and an assignment phase. In the prediction phase, various learning models are integrated to predict the future locations and routes of workers, while the assignment phase employs a graph decomposition-based optimization algorithm to allocate group tasks. Meanwhile, pioneering works^{29,30} have used reinforcement learning-based methods to solve group task allocation problems. For example, Tong et al.²⁹ studied the large-scale vehicle scheduling problem and proposed a method that combines combinatorial optimization and reinforcement learning to optimize the total benefit of the overall allocation scheme. Wang et al.³⁰ studied the task assignment problem of data collection in human-machine collaborative spatial crowdsourcing and proposed a distributed multi-agent reinforcement learning method.

Existing research on group task assignment encompasses various categories of methods, each with its unique strengths and application scenarios. Among them, the application of reinforcement learning in these studies provides the advantage of dynamically optimizing and large-scale task assignment, allowing for adaptive decision-making in diverse and evolving environments.

Path planning in spatial crowdsourcing

Path planning in logistics warehouses is significant for optimizing logistics operations and improving efficiency, aiming to find the optimal path for robots to deliver racks quickly, which is also closely related to path planning in spatial crowdsourcing. We categorize and review existing path planning approaches, categorizing them into traditional methods and reinforcement learning-based methods to provide a comprehensive overview of the field.

Traditional methods are usually simple to describe and easy to implement. The pioneering work³¹ was the first to conceptualize each AGV as a drive-unit agent and apply the A* algorithm for path planning in warehouse.

Subsequently, research on path planning in warehouses has focused on two primary objectives: ensuring conflict-free routes and mitigating congestion. For expmple, many existing works^{11,15,16,23} consider path planning as a *Multi-Agent Path Finding (MAPF)* problem, focusing on finding conflict-free routes for all AGVs to complete assigned tasks efficiently. Specifically, Shi et al.¹¹ propose a variant of the A* algorithm to plan AGV routes, incorporating time factors to effectively avoid conflicts. To reduce the time complexity of the dynamic path planning algorithm, Tong et al.³² proposed a flexible multi-objective path planning algorithm, which can solve the path planning problem of inserting online requests and prediction requests. Luo et al.³³ designed a diversified optimal *k* path finding algorithm with efficient shortest path tree splicing, constant similarity comparison, and candidate path pruning. Although these traditional methods can effectively improve path planning efficiency in specific scenarios, they still have certain limitations when dealing with more complex dynamic systems or real-time optimization situations due to the computational complexity.

As an interactive “trial-and-error” learning technology, reinforcement learning can effectively solve sequential decision and path planning problems by dynamically modeling the environment. For example, Zhang et al.³⁴ combined path planning with energy management to construct a dual-loop plug-in hybrid vehicle and power management control scheme. Sartoretto et al.³⁵ proposed a multi-agent path-finding algorithm called PRIMAL, which uses a distributed model to increase the number of agents in large-scale path planning scenario. Liu et al.³⁶ used an image feature extraction method for modeling and proposed a distributed multi-agent reinforcement learning algorithm with an evolutionary training method, which improved the robustness of large-scale robot path planning training in complex environments. While these works provide online path planning for warehouses, their solutions ignore the influence of possible congestions, and thus still cannot solve the RTTP problem well.

Table 1 summarizes the key differences between our approach and existing works. However, we argue that prior studies are ill-suited for online task planning in RTTP problem due to the following limitations:

- *Limitation 1:* Prior studies^{9,11} typically address order assignment and path planning as separate sub-problems, overlooking their interdependence and the potential benefits of joint optimization. This disjointed approach may result in decisions that negatively impact the overall efficiency of order processing.
- *Limitation 2:* Existing studies^{11,23} adopt the A* algorithm for dynamic path planning, which often results in high time complexity. With a large of items flushing in, the overall time complexity becomes impractical for real-time execution.
- *Limitation 3:* In general, the existing works^{12,15,16} provide conflict-free routes for AGVs in warehouses, yet they ignore possible congestions, which greatly affects the work efficiency of AGVs⁹.In response, we adopt a holistic approach to problem formulation. This study addresses the task planning challenges in logistics warehousing by investigating the joint optimization of order assignment and path planning. The primary objective is to develop heuristic joint optimization algorithms and reinforcement learning-based methods to enhance the efficiency of warehouse order picking. To lay the groundwork for these solutions, we formally define the RTTP problem in the following section.

References	Task	Optimization objective	Task assignment	Path planning	Online	Scenario
9	Route planning and task assignment	Minimize the travel cost and tardiness cost	Matching-based	THA* -based	✓	One-stop warehouse service
11	Adaptive task planning	Minimize the end-to-end makespan	Q-Learning-based	A* -based	✓	Task planning in robotized warehouse
12	Collision-aware route planning	Minimize path costs while avoiding collisions	✗	Segment-based	✓	Multi-robot system in warehouse
20	Order batching and pick order assignment	Maximize the total throughput	Heuristic-based	✗	✗	Robotic mobile fulfillment system
22	AGV path planning	Minimize path costs	✗	PSO-based	✗	Path planning in warehousing
23	Multi-Agent Path Finding	Minimize path costs while avoiding collisions	✗	A* -based	✓	Non-highly controlled warehousing
24	Path planning with high degree of freedom	Minimize path costs while avoiding collisions	✗	MPC-based	✓	Path planning in warehouse
26	Utility-aware ride matching	Maximize the total utility	Matching-based	Insertion-based	✓	Dynamic ridesharing in spatial-crowdsourcing
Ours	Joint optimization of task assignment and path planning	Minimize travel costs and order completion time	MARL-based multi agent reinforcement learning	THA* -based	✓	Task planning in intelligent warehousing

Table 1. Comparison of existing task planning solutions. • “Matching-based” indicates modeling task assignment as a bipartite graph matching problem. “Q-Learning-based” indicates that the corresponding algorithm is designed based on Q-Learning. “Heuristic-based” means designing the algorithm based on heuristic rules. “PSO-based” represents the particle swarm optimization algorithm. “A* -based” indicates that the path planning method is designed based on A* algorithm. “Segment-based” indicates that the path planning method is designed based on the segmentation of the path into straight-line segments. “MPC-based” indicates that the algorithm is designed based on model predictive control. “Insertion-based” refers to a path planning method that constructs the path by progressively inserting intermediate waypoints to improve the overall route. • “Online” indicates whether the solution can deal with the real-time tasks in the dynamic environment

Problem definition

In this section, we first introduce the system model and basic concepts, followed by a formal definition of the RTTP problem.

System model

As shown in Fig. 1, in the logistics warehouse under RMFS, first, pickers and racks are located at fixed positions in the picking area and rack storage area in the warehouse, respectively. Each rack contains multiple types of items, and a specific type of item can be scattered across several racks. Then, when there are orders, the robots will move the relevant racks to the picker for picking. Pickers pick out the relevant items of an order from the racks and put them into a bin. If an order is thoroughly picked, the bin is placed on a conveyor belt and then packed and shipped. Furthermore, due to the limitation of rack capability and the number of bins, the number of orders a picker can pick simultaneously is limited. Then, we formally give the definitions of those involved in this paper.

Basic definitions

Definition 1 (Order) An order o is represented as a 3-tuple $\langle a, d, \Psi_n \rangle$, where $o.a$ and $o.d$ represent the arrival time and the expiration time of the order, respectively. $o.\Psi_n$ represents the set of items required by the order. The lifespan of an order is limited by its arrival and expiration times, indicating that orders nearing their expiration time must be prioritized for execution. We assume that orders arrive in streams, denoted by O .

Definition 2 (Item) An item g is represented as a 2-tuple $\langle t, u \rangle$, where the item's type is $g.t$, and the processing time for this item is $g.u$.

The processing time $g.u$ of item g refers to the time required for a picker to handle the item at the workstation, i.e., the time for the picker to take it from the rack and put it into the corresponding order bin.

Definition 3 (Rack) A Rack r can be represented as $\langle l, s, \Psi, \hat{\Psi} \rangle$, in which $r.l$ is the rack's real-time location in the storage area, $r.s$ indicates the state of the rack. $r.\Psi$ is the set of items contained on this rack, and $r.\hat{\Psi}$ is the set of items that need to be currently delivered from rack r to the picker.

In the robotized warehouse, each rack contains multiple items, and a single class of items may be stored across multiple racks. Each rack can be sent to any workstation for processing according to the order's requirements. At the workstation, the picker only needs to retrieve the required items from the rack, pick them, and place them into the corresponding order bin. We use R to denote the set of all racks.

Definition 4 (Robot) A robot $h \in H$ is represented as $\langle l, s, e \rangle$ with real-time location $h.l$ and state $h.s$, and $h.e$ is the current remaining power of the robot.

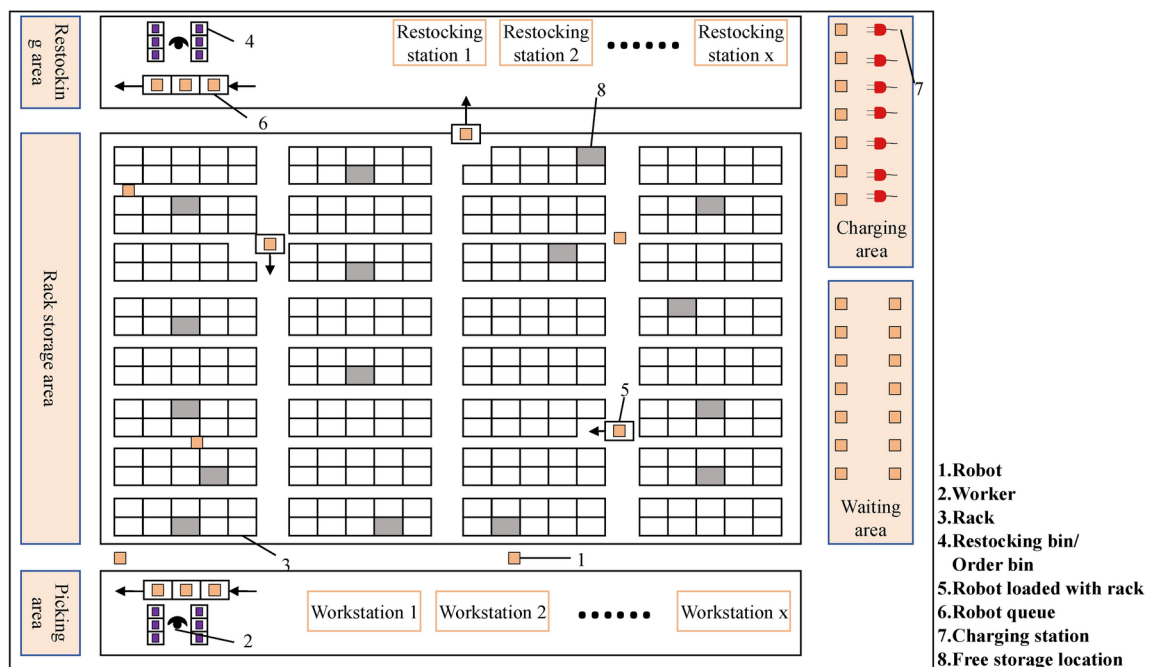


Figure 1. Logistics warehousing layout and picking process.

Since the robot is mobile, its location, state, and remaining power change over time. The state $h.s$ can be either “busy” or “idle”. A robot is considered as “busy” if it is engaged in rack pickup, delivery, queuing, or processing; otherwise, it is “idle”. For simplicity, we assume that the robot moves at a constant speed, and its power consumption increases linearly with time. This assumption can be replaced with a more practical model without affecting the core problem of this study, making it a reasonable simplification. When the robot’s power drops below a threshold δ , it needs to stop working and move to the nearest charging station. To ensure that the robot can always reach a charging station from any location in the warehouse, a global and uniform threshold δ can be set. In real-world scenarios, the robot can be any suitable form of Automated Guided Vehicle (AGV).

Definition 5 (Workstation) A workstation w is a 4-tuple $\langle l, p, Q, B \rangle$, in which $w.l$ is its fixed location and $w.p$ is the estimated remaining processing time of the currently picked rack. $w.Q$ is the queue of racks waiting to be processed. $w.B$ is the set of order bins for the workstation.

We assume that a workstation processes the items on the racks in the queue using a “first-come-first-serve” approach. This assumption is reasonable because the picking area is typically limited, making it difficult for robots carrying racks to bypass others. Additionally, the time required for a picker to retrieve an item from a rack is fixed.

Definition 6 (Order Bin) An order bin $b \in w.B$ is represented as $\langle o, \Psi_{re} \rangle$, where o represents the order currently corresponding to the order bin, and the remaining items required are Ψ_{re} .

The workstation contains a certain number of bins, with each bin corresponding to a specific order. Order processing is considered complete when all the order items are put into the bin. Once filled, the bin is placed onto the conveyor belt for the subsequent packaging operation. Meanwhile, a new empty bin is placed in the workstation’s original position to handle other orders.

Definition 7 (Makespan) The Makespan M is the time from the match of the first order till the pick completion of the last order.

Assuming the first order matches at time 0, M equals the time when the last order is completed, which can be calculated as:

$$M = \max_{r \in R} f_r \quad (1)$$

where f_r denotes the latest time when the rack r finishes picking. It can be calculated as $f_r = t_e + t_m + t_q + t_p$, where t_e is the last time the rack is selected, and the remaining three terms correspond to the delays for rack delivery, queuing, and processing. The calculation formulas for t_m , t_q , t_p are as:

$$t_m = t(v.l, r.l) + t(r.l, w.l), \quad t_q = \max\{t_m - f_w, 0\}, \quad t_p = \sum_{g \in r.\Psi} g.u \quad (2)$$

$$f_w = w.p + \sum_{r \in w.Q} \sum_{g \in r.\Psi} g.u \quad (3)$$

Here, f_w represents the delay incurred by the picker in the workstation to process the items on all the racks in the queue. As such, we can formally define the Real Time Task Planning (RTTP) problem.

Real time task planning

Definition 8 (RTTP Problem) Given a set of racks R , a stream of real-time orders O , and a set of robots H located within the warehouse, the RTTP problem involves comprehensively selecting the appropriate racks based on the orders’ requirements and the current locations of idle robots, with the goal of minimizing the makespan M .

In general, orders that arrive in a stream are usually processed in batches^{9,11,12}, where the order stream is divided into batches and the task planning is performed within each batch. As such, to analyze the difficulty of the RTTP problem, we prove that the task planning problem in each batch (we term batch-based RTTP problem) is NP-hard.

Theorem 1 The batch-based RTTP problem is NP-hard.

NP-hard (Non-deterministic Polynomial-time hard) refers to a class of computational problems that are at least as difficult as the hardest problems in NP

Proof Two aspects primarily influence the batch-based real time task planning: 1) calculating path costs when selecting racks and 2) the outcomes of order matching. Consequently, this problem can be regarded as an order matching problem with constraints. Subsequently, we reduce the batch-based RTTP problem to the Set Cover Problem (SCP), a classical NP-hard problem. Given the universal set $\mathcal{U} = \{u_1, u_2, \dots, u_n\}$, and its m subsets $\mathcal{S} = \{s_1, s_2, \dots, s_m\}$, where $s_i \in \mathcal{U}$ & $\bigcup_{j=1}^m s_j$, set coverage refers to finding a set of sets $\bigcup_{j \in I} s_j = \mathcal{U}$, where $I \in \{1, 2, \dots, m\}$. As such, given a pair $(\mathcal{U}, \mathcal{S})$ and an integer k , the SCP determines whether a covering outcome can be found with the number of sets not exceeding k .

In the batch-based RTTP problem, the set of orders is regarded as the universal set $\mathbb{U}$. In contrast, the rack set constitutes the elements of $\mathbb{S}$ to identify the optimal rack combination that minimizes the order completion time. Given that the order completion time is influenced by the order processing efficiency, which in turn is contingent upon the matching outcomes and path planning, under the premise of a well-defined path planning algorithm, the path cost is treated as the weight influencing rack selection, thus transforming shelves into a weighted subset of a set. Consequently, the rack matching outcomes solely govern the order processing efficiency. The quest for the rack combination that minimizes the order completion time is tantamount to solving the minimum weighted set cover problem. Therefore, the optimization problem can be framed as a weighted set cover problem. Since the weighted set cover problem falls under the broader category of set cover problems, and set cover problems are known to be NP-hard, the batch-based real time task planning problem is also NP-hard. $\square$

Solution

In general, the warehouse offers centralized services in a batch processing manner, whereby the warehouse computes an evaluation value for each rack to execute a specific order, allocates the order to appropriate racks based on this evaluation value, and then instructs the robot to move the rack to execute the corresponding order. Consequently, a joint optimization evaluation indicator is first proposed to assess the suitability of a rack for executing an order, followed by an order assignment algorithm that assigns orders to suitable racks.

Joint optimization evaluation indicator

The joint optimization evaluation indicator serves as one of the core components in addressing the RTTP problem, with its comprehensive consideration aimed at optimizing order processing time. Adopting an effective rack selection plan enhances the efficiency of rack movements, minimizes travel distances, and thereby facilitates the swift completion of order processing. This indicator enables the assessment of moving a particular rack within the current warehouse environment. The greater the evaluation indicator value, the more significant its contribution to fulfilling the current order, considering the factors of rack content and location.

Overall, the value of evaluation indicator is influenced by two factors simultaneously: matching degree and path cost. Firstly, the matching degree is considered, which refers to the extent of alignment between the rack and the current order. Specifically, it is manifested as the ratio of the number of items in order on the current rack to the total number of items in the order. A rack selection plan with a high matching degree implies the capability to process more items within the same period, thereby accelerating the order picking process. The matching degree between rack r_j and order o_{j*} is calculated as follows:

$$Match(r_j, o_{j*}) = \frac{\sum_{k=1}^Q |r_j \cap o_{j*,k}|}{\sum_{k=1}^Q |o_{j*,k}|} \quad \text{iris dari rack ke order itu ada brapa item} \quad (4)$$

Meanwhile, the path cost is introduced, indicating the cost of moving the rack. Specifically, the path cost of the rack r_j can be rapidly computed using the B* algorithm, which considers the presence of obstacles in the environment, and it can be represented as $P_{r_j} = B^e(r_j)$. The calculation function for the joint optimization evaluation indicator $E_{r_j, o_{j*}}$ is given by:

$$E_{r_j, o_{j*}} = \alpha_{r_j, o_{j*}} \left[w_1 \times Match(r_j, o_{j*}) + w_2 \times \frac{1}{P_{r_j}} \right] \quad (5)$$

$$\alpha_{r_j, o_{j*}} = Match(r_j, o_{j*}) \cap P_{r_j} \quad (6)$$

where $Match(r_j, o_{j*})$ and P_{r_j} are both normalized to $[0, 1]$, w_1 and w_2 are two hyperparameters used to balance the effect of matching degree and path cost. The $\alpha_{r_j, o_{j*}}$ indicates that the joint optimization evaluation indicator is meaningful only when both the rack location and the items required by the order in the rack are considered simultaneously. By selecting racks with a high matching degree to the order and minimizing path costs, the system can fulfill order requirements more efficiently, reducing waiting times and optimizing overall order processing.

Joint optimization task planning (JOTP)

Based on the joint optimization evaluation indicator, we propose a joint optimization task planning algorithm that identifies the optimal matching between racks and orders within each batch. Despite the real-time arrival of orders, the warehouse offers batch processing services for more efficient sorting¹¹. In each batch, all orders and racks capable of executing tasks are first gathered. Subsequently, the warehouse employs the order assignment algorithm to allocate the orders to the most suitable racks.

Figure 2 shows the general framework of joint optimization task planning. Specifically, there are three orders (o_1, o_2, o_3) and eight racks ($r_1, r_2, \dots, r_8$) in different positions containing items that can partially fulfill the orders. Here, the racks associated with order o_1 are r_1, r_3 , and r_4 ; Order o_2 is related to racks r_2 and r_3 ; Order o_3 is associated with rack r_3 . The order assignment algorithm gives the plan that orders o_1 are fulfilled by racks r_1 and r_3 , rack r_2 fulfills order o_2 , and rack r_3 fulfills order o_3 . The robot closest to each rack is selected for rack delivery based on the assignment results. Since the robot can move directly from beneath the rack when unladen, its route to the rack can be planned as a straight line.

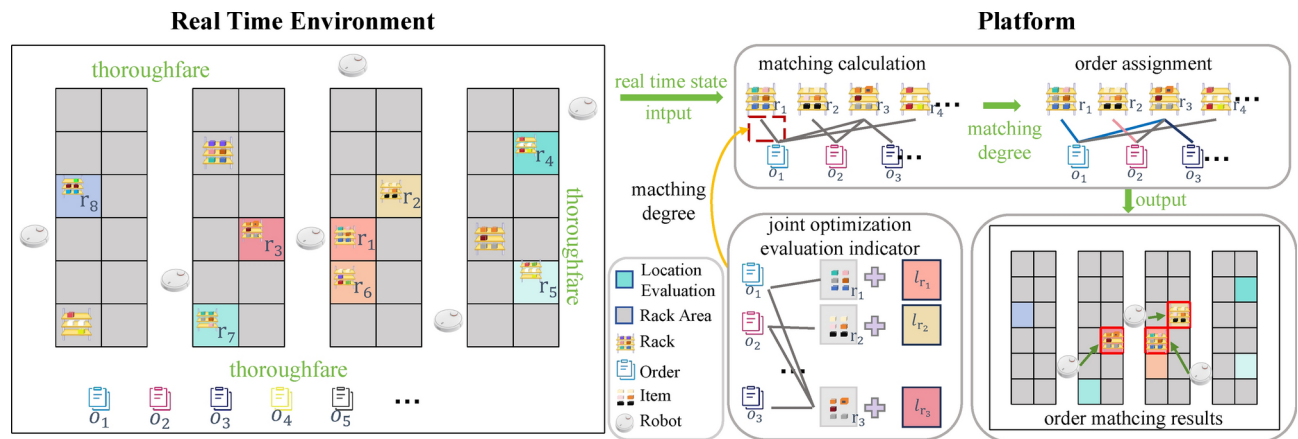


Figure 2. Framework of joint optimization task planning.

Next, we give the detailed pseudo code of the joint optimization task planning (JOTP) algorithm, illustrated in Alg. 1. Specifically, it takes a set of racks R and orders O as input and outputs the order assignment plan M . It first initializes the order matching plan M and rack-order bipartite graph G as $\emptyset$ (line 1). Then, it traverses all combinations of racks and orders and calculates the value of $c(r, o)$ according to Equ. 5 (lines 2-3). If $c(r, o) > 0$, it adds $(r, o, c(r, o))$ as an edge to graph G (lines 4-5). Next, it calls KM algorithm^{37,38} to get the maximum weight matching on the bipartite graph G , adds the results M' into M and updates G according to M' (i.e., remove the edges from graph G related to the racks and orders in matching plan M') until there are no edges in G (lines 6-9). Finally, it returns the optimal matching plan M (line 10).

khun munkres algorithm (hungarian)

Input: A set of racks R and a set of orders O

Output: The optimal rack-order assignment plan M

```

1 Initialize the order assignment plan  $M$  and rack-order graph  $G$  as  $\emptyset$ ;
2 foreach  $(r, o) \in R \times O$  do
3    $c(r, o) \leftarrow$  calculate the evaluation indicator value according to Equ. 5;
4   if  $c(r, o) > 0$  then
5     add  $(r, o, c(r, o))$  into graph  $G$ ;
6 while there are still edges in  $G$  do
7    $M' \leftarrow$  call the KM algorithm37,38 to get the maximum weight matching on graph  $G$ ;
8    $M \leftarrow M \cup M'$ ;
9   update  $G$  according to  $M'$ ;
10 return  $M$ ;

```

Algorithm 1. Joint Optimization Task Planning algorithm (JOTP)

Multi-agent joint optimization task planning based on RL (JOTP-RL)

The JOTP algorithm, as described above, leverages the similarity between orders and items on racks. By transforming the order-picking problem in logistics warehouses into a weighted set cover problem, it effectively addresses the order assignment challenge. To further improve the quality of task planning, we propose a multi-agent joint optimization task planning algorithm based on reinforcement learning (JOTP-RL). Multi-agent reinforcement learning relies on the collaboration among agents, where each agent functions as an independent decision-making entity. Through interactive learning with the environment, each agent optimizes its strategy to maximize the collective reward associated with the common goal. To address the joint optimization task planning problem in logistics warehousing, we propose a multi-agent reinforcement learning decision-making framework based on the Q-Mix algorithm.

Overall workflow

In general, the rack selection process is modeled as a partially observable Markov decision process, and we employ the Q-Mix algorithm³⁹ to solve it, where racks are regarded as a group of homogeneous agents. Upon the arrival of an order, these agents make decisions by considering both the current global environment and their individual local states. Through mutual communication and collaborative decision-making, they collectively determine the optimal set of racks for efficiently executing the current order, ensuring that the solution aligns with both global objectives and local constraints. Subsequently, the idle robots are assigned to move the selected racks to the corresponding workstation and return to their original positions after order picking is completed.

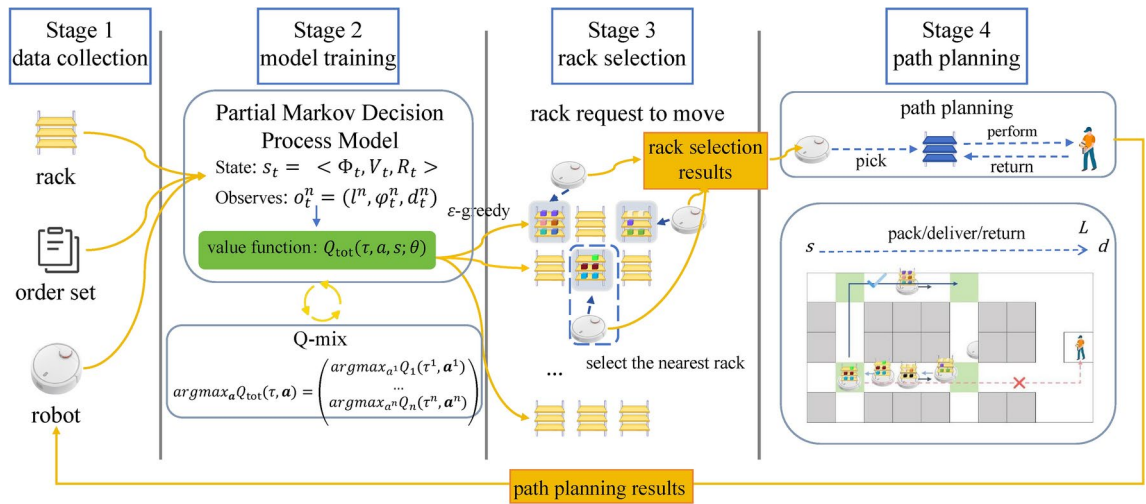


Figure 3. Workflow of the multi-agent joint optimization task planning.

The workflow of the multi-agent joint optimization task planning can be summarized into four steps, as shown in Fig. 3.

Stage 1: Data Collection. This stage involves collecting real-time data such as order requirements, rack state, and robot location. The agent understands the current state of the environment through such real-time spatiotemporal data.

Stage 2: Model Training. The Q-mix algorithm trains the agents to learn the collaboration strategy among agents and the best response to the current order, which aims at enabling the order-picking system to achieve better performance.

Stage 3: Rack Selection. Each agent can make the optimal rack movement decision based on the current environmental state through model training. The result of their choices is a set of movable racks, which are used to fulfill the current order.

Stage 4: Path Planning. After getting the rack movement decision results, planning the paths for these racks is required. As such, we propose the THA* algorithm to plan paths for the corresponding robots to optimize the overall task completion time.

Partially observable Markov decision process

The modeled partially observable Markov decision process can be represented by a 7-tuple $\langle F, S, A, O, P, R, \gamma \rangle$. Given the decision time $t \in \{1, 2, \dots, T\}$, all agents make a joint action based on their respective observed states at each decision time step t . Next, we provide a detailed introduction to the modeled Markov decision process.

Agent. $F = \{r_1, r_2, \dots, r_N\}$ is the set of N homogeneous agents representing all racks in the warehouse.

State. $S = \{s_1, s_2, \dots, s_t\}$, $s_t \in S$ represents the global state of the warehouse environment at time step t . Specifically, the global state can be defined as a triple (Φ_t, V_t, R_t) , where Φ_t represents all orders in the warehouse up to time step t , V_t represents the location of all idle robots, and R_t represents all racks in the warehouse including the items on the racks and location of the racks at time step t .

Actions. $A = \{A_1 \times A_2 \times \dots \times A_N\}$ represents the joint action space. $a_t \times a_t^2 \times \dots \times a_t^n$ is defined as the joint action of racks in R at the time step t , where $a_t^n \in A_n$ represents the action of rack r_n . Specifically, $a_n \in \{0, 1\}$, where 0 and 1 represents whether rack r_n is selected to fulfill orders at time step t .

Observation. $O = \{O_1 \times O_2 \times \dots \times O_N\}$ represents the joint local observations of the agents. At the time step t , the local observation value of the rack r_n is defined as $(l^n, \phi_t^n, d_t^n) \in O$, where l^n represents the initial position of the rack, ϕ_t^n represents the order matched to the rack, and d_t^n represents the real-time position of r_n .

State Transition. The local observation transfer function of the rack is $P(o_{t+1}|a_t, o_t)$, and the global state transfer function of the warehouse is $P(s_{t+1}|a_t, s_t)$. Also, by employing the centralized training with decentralized execution (CTDE) framework^{40,41}, the observation state of each agent can be communicated and shared during model training stage. In the distributed execution stage, each rack decides whether to move based on its local observations.

Reward. R represents reward function. The optimization goal of task planning in logistics warehousing is to minimize the completion time, and two main factors affect the efficiency of task planning: i). The number and location of the selected racks. ii). The efficiency of rack movement. Therefore, we design the reward function by considering three parts: rack movement reward, decision-making conflict-free reward, and order picking completion reward. $r_t = r_t^e + r_t^o + r_t^h$, where $r_t^e = E^{R_s} - \sum_{r \in R_s} B(r)$ represents the reward for the moving distance when the rack is selected to move. Here, E^{R_s} represents the maximum estimated sum of the moving distances of the selected racks, $B(r)$ represents the estimated distance for the rack to complete the task, and R_s represents the set of racks selected. r_t^o is designed to avoid the selection of useless racks:

$$r_t^\sigma = \begin{cases} \sum_{r \in R} r_c, & \text{the rack is selected and matched to an order} \\ 0, & \text{the rack is selected but not matched to an order} \end{cases} \quad (7)$$

then, the target reward r_t^h obtained after completing the order-picking task is indicated to improve the efficiency of the set of racks in completing the picking task as a whole.

Discount Factor. γ indicates whether the model focuses on current rewards or long-term rewards.

We then apply Q-Mix, a multi-agent reinforcement learning algorithm⁴², to solve the partially observable Markov decision process. Indeed, Q-Mix has the following advantages in solving the task planning problem. Firstly, Q-Mix effectively tackles partial observability in the environment by rationally integrating global and local information, enhancing each agent's comprehension of the overall warehouse state and decision-making accuracy. Secondly, by incorporating a centralized mixing network, Q-Mix enables inter-agent coordination in decision-making, accommodating the complexity of the warehouse environment. This elevates the overall efficiency of the system. Furthermore, applying Q-Mix imparts greater adaptability and generalization capabilities to the system, enabling it to handle dynamic warehouse environments and ensure robust performance across diverse scenarios.

Optimization of path planning

The task planning process described above includes both order assignment and path planning. After deciding on the racks to be moved, we need to plan paths for robots so that the racks can be efficiently moved to the workstation. Typically, A* algorithms are employed for path planning in warehouse^{11,12,43}. However, when handling a large number of robots, the A* may have two main limitations: first, it calculates paths for many locations that are ultimately irrelevant, and second, it does not consider congestion, which results in increased tardiness costs. Specifically, much time is wasted as robots attempt to avoid collisions, leading to inefficiencies. To address these issues, we introduce a novel time-dependent hybrid approach (THA*)⁹ by considering the warehouse's regularity for path planning to further enhance efficiency and effectiveness.

By integrating the order assignment method based on multi-agent reinforcement learning and the path planning algorithm based on THA*, the overall process of JOTP-RL is obtained, as shown in Algorithm 2.

Input: A set of racks R , a set of orders O , a set of robots H , and time step t

Output: The decision planning scheme at time t .

1 Initialize:

2 Initialize the environment, including racks set R_t , order set O_t , free robot set H_t .

3 Initialize network parameters, including learning rate, discount factor, experience cache size, etc.

4 Order Assignment Step:

5 Execute the Q-mix distributed execution algorithm to obtain the joint action.

6 while $t = 1$ to T do

7 while $r = 1$ to N do

8 Load the agent network and parameters θ_n ;

9 Take X_t^n as the network input and select the action a_t^n ;

10 organize joint action $a_t^{mix} = \{a_t^1, a_t^2, \dots, a_t^n\}$ and update the action

11 Path Planning Step:

12 call the THA* algorithm to plan the optimal path for robots

13 **return** Decision-making planning at time t .

Algorithm 2. Joint Optimization Task Planning algorithm based on RL (JOTP-RL)

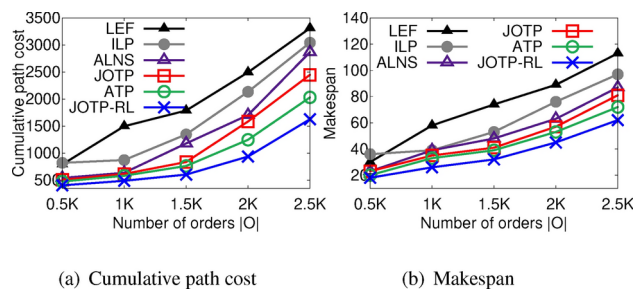
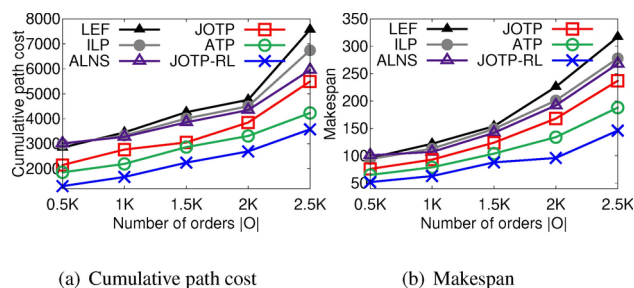
Experiments

We conducted extensive experiments on two datasets to validate the effectiveness of the proposed algorithms.

Experimental settings

Dataset and parameter settings. In the joint optimization evaluation indicator of JOTP, we set the two hyperparameters, w_1 and w_2 , both to 0.5, thereby ensuring that the matching degree and path cost have equal influence and effect. For the parameter settings of the JOTP-RL algorithm, we fine-tuned the hyperparameters based on insights from existing work presented in Section 2. The best-performing parameter set was adopted by balancing solution quality and computational efficiency, thereby optimizing the trade-off between accuracy and computational time. During the training stage, we set the number of epochs to 5000 and the batch size to 32. The discount factor, γ , was set to 0.999, and the experience cache size was 5000. For the learning rate, we adopted the default setting from Q-Mix⁴², i.e., $lr = 5e - 4$. Adam was used as the network optimizer, and ReLU as the activation function. Table 2 provides a description of the dataset and the key parameter settings varied in the experiments. We generally set the parameters based on⁹ and adjusted them according to our survey of a local

Parameter	Dataset	
	Syn-A	Syn-B
The average number of items per order	3 (items)	8 (items)
The number of racks	24 (units)	48 (units)
The number of item types	30 (kinds)	60 (kinds)
The number of orders $ O $	500,1000, 1500 ,2000,2500 (pieces)	500,1000, 1500 ,2000,2500 (pieces)
The maximum number of item types in warehouse $ C $	7,9, 11 ,13,15 (kinds)	22,26, 30 ,34,38 (kinds)
The maximum number of items in a rack $ Q $	5,7, 9 ,11,13 (units)	9,11, 13 ,15,17 (units)

Table 2. Dataset and parameter settings.**Figure 4.** Effect of the number of orders (Syn-A).**Figure 5.** Effect of the number of orders (Syn-B).

company's unmanned logistics warehouse. The numbers in **bold** represent the default parameter values used in these experiments.

Evaluation metric. Regarding the effectiveness of algorithms, we employ two metrics: the makespan M (defined in Def. 7) and cumulative path cost (CPC), which can be calculated as $\sum_{r \in R} (r.end - r.start) \cdot c_t$. Here, $r.start$ and $r.end$ represent the start time and end time for rack r to perform a specific task, respectively. c_t represents the moving cost per unit of time. The experimental results are an average of 10 runs.

Compared algorithms. We compare the proposed algorithms, JOTP and JOTP-RL, with the following four algorithms. The parameters for these algorithms are set according to the guidelines provided in the original literature to ensure optimal performance.

- LEF⁴⁴: selects the task with the shortest expiration time. In logistics warehousing, all orders are to be executed. Consequently, assuming each order incurs an equal degree of delay, it selects the rack for the earliest appearing order.
- ILP⁴⁵: proposes an integer linear programming method to process orders of items from different racks.
- ALNS⁴⁶: is the state-of-the-art approach that finds feasible plans between orders and racks by iteratively utilizing destruction and repair operators along with an integrated local search.
- ATP¹¹: considers the trade-off between queuing time and movement time for racks in large-scale scenarios, and determines the rack to be moved at each moment from the perspective of a single agent.

Experimental results

Effect of the number of orders. As shown in Figs. 4 and 5, there is an upward trend in both makespan and cumulative path cost across all algorithms, as the number of orders increased in both Syn-A and Syn-B datasets. The LEF algorithm exhibited the poorest performance due to its simplistic first-come-first-service processing

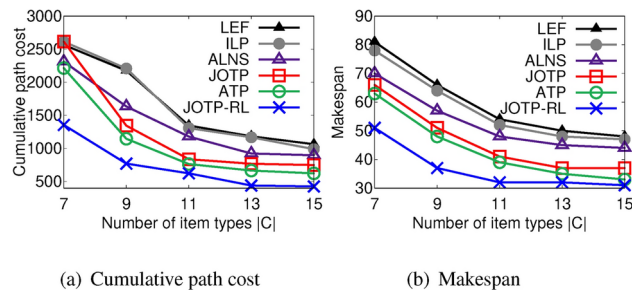


Figure 6. Effect of the number of item types (Syn-A).

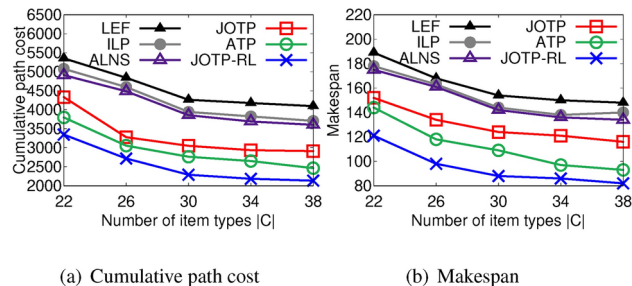


Figure 7. Effect of the number of item types (Syn-B).

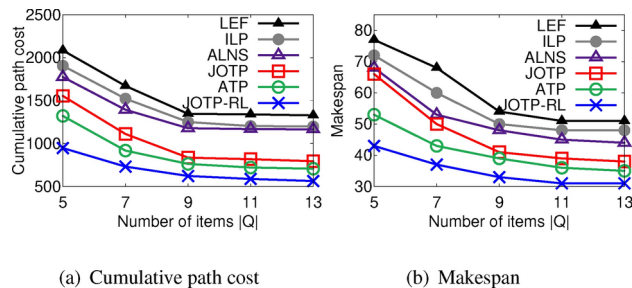


Figure 8. Effect of the number of items (Syn-A).

strategy. In contrast, the ILP algorithm surpassed LEF by comprehensively considering constraint conditions and optimization objectives. The ALNS and JOTP algorithms perform better for large-scale cases, with JOTP further enhancing efficiency by accounting for the impact of rack movement. Notably, the reinforcement learning algorithm JOTP-RL stood out, leveraging a multi-agent reinforcement learning approach to select optimal racks, substantially reducing makespan and cumulative path cost, thereby achieving the best performance. This advantage was particularly pronounced when handling large-scale orders, validating the robust adaptability of our JOTP-RL.

Effect of the number of item types. As shown in Figs. 6 and 7, our experimental results demonstrate that, as the number of item types increases, both the makespan and cumulative path cost generally decrease across all algorithms, eventually stabilizing after reaching a certain threshold (e.g., $|C| = 11$). The JOTP-RL algorithm, leveraging a reinforcement learning-based multi-agent task planning framework, achieves significant reductions in both makespan and cumulative path cost, particularly when handling orders with a high degree of item type diversity. This performance aligns with the optimization objectives established in the paper, highlighting the effectiveness of the proposed approach in real-time scenarios. The JOTP algorithm, which utilizes a batch-based joint optimization evaluation indicator, also exhibits notable improvements and outperforms traditional methods. However, it lags behind JOTP-RL, especially when dealing with orders that contain a diverse set of item types. In contrast, the ATP algorithm performs moderately across varying numbers of item types, exhibiting only slight reductions in makespan and cumulative path cost compared to JOTP and JOTP-RL. This reflects the limitations of the ATP method in handling the complexities associated with diverse item types and the dynamic nature in real-time task assignment. Overall, the integration of joint optimization task assignment and path planning, enhanced by the improved THA^* algorithm for efficient path planning, allows the JOTP-RL

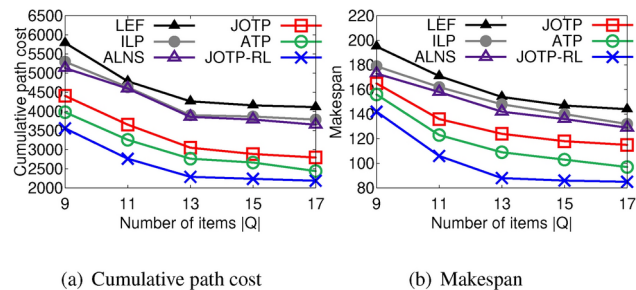


Figure 9. Effect of the number of items (Syn-B).

Datasets	Order number	LEF	ILP	ALNS	ATP	JOTP	JOTP-RL
Syn-A	500	0.12	1.38	3.72	0.58	0.95	1.11
Syn-A	1500	0.36	3.82	11.54	1.92	3.17	3.51
Syn-A	2500	0.58	7.56	21.84	3.45	6.08	6.67
Syn-B	500	0.15	1.62	3.91	0.65	1.03	1.51
Syn-B	1500	0.42	4.59	12.02	2.11	3.43	4.26
Syn-B	2500	0.71	8.03	22.65	3.91	6.54	7.48

Table 3. Average computational times (in seconds) for different algorithms across order numbers.

Datasets	Order number	LEF	ILP	ALNS	ATP	JOTP	JOTP-RL
Syn-A	500	256	912	789	1024	634	501
Syn-A	1500	328	1364	1120	1500	1024	868
Syn-A	2500	487	1800	1550	2056	1483	1234
Syn-B	500	220	878	688	980	612	470
Syn-B	1500	280	1140	1001	1320	980	822
Syn-B	2500	396	1650	1350	1780	1280	1045

Table 4. Comparison of peak memory usage (in MB) across different algorithms.

algorithm to outperform existing approaches, delivering superior performance in both execution time and task optimization.

Effect of the number of items. Figures 8 and 9 illustrate the impact of varying the upper bound of item’s quantity per rack on the efficiency of task planning. Generally, as $|Q|$ increases, these metrics exhibit a trend from significant decline to gradual stabilization. Expressly, when $|Q|$ is small, the insufficiency of the item necessitates an increase in the number of racks utilized, leading to marked increases in makespan and cumulative path cost. However, once $|Q|$ exceeds a certain threshold (e.g., $|Q| = 11$), the gains in efficiency from further increasing the quantity of the items gradually diminish, highlighting the limitations of solely relying on quantity augmentation to enhance the efficiency of task planning. Among all algorithms, the JOTP-RL shows optimal performance in most cases. In contrast, other algorithms also exhibit similar change trends.

Analysis of Computational Times. To evaluate the computational times of different algorithms under various order numbers, we conducted a set of experiments on two datasets with order numbers ranging from 500 to 2500. The experimental results of the average computational time (in seconds) are shown in Table 3. From the results, it is evident that simpler heuristics like LEF and ATP exhibit significantly lower computational times, making them more suitable for scenarios requiring quick solutions. However, more advanced methods like JOTP and JOTP-RL, while computationally more intensive, achieve superior task allocation performance, as demonstrated in our main results. Notably, JOTP-RL exhibits a moderate increase in computational time compared to JOTP due to the reinforcement learning component, which enhances decision-making precision.

Analysis of Memory Consumption. To further demonstrate the superiority of our algorithm, we conducted a set of experiment to evaluate memory consumption of different algorithms under various order numbers. In this experiment, we track the peak memory usage (in MB) of each algorithm as it processes different datasets. The experimental results as shown in Table 4, JOTP-RL consistently demonstrates lower peak memory usage across different datasets compared to ILP, ALNS, ATP, and JOTP. For instance, for the Syn-A 2500 dataset, JOTP-RL uses 1234 MB, which is significantly lower than ILP’s 1800 MB, ALNS’s 1550 MB, and ATP’s 2056 MB. The results indicate that JOTP-RL not only provides higher quality solutions, but also achieves better memory efficiency compared to other algorithms, especially as the order increases. The memory efficiency of JOTP-RL makes it a more practical solution for real-time task allocation in resource-constrained environments.

Summary and Discussion. Our experiments yielded several important insights into real-time task planning for unmanned logistics warehouses:

(1) **Algorithm Performance:** The JOTP-RL algorithm, leveraging multi-agent reinforcement learning (MARL), significantly outperforms existing methods in terms of both makespan and cumulative path cost, especially in large-scale scenarios with high item type diversity. This improvement directly stems from the algorithm's ability to adapt to real-time, dynamic environments, facilitated by agent collaboration and interactive learning with the environment within the multi-agent reinforcement learning-based task planning framework.

(2) **Impact of Item Types:** As the number of item types increases, both makespan and cumulative path cost initially decrease, reaching a point of stabilization. Furthermore, adding more items per rack improves overall efficiency, though this effect plateaus after a certain threshold. These results align with the objective of optimizing task allocation and path planning in real-time environments, as described in the paper.

(3) **Comparison with Existing Methods:** While JOTP-RL involves higher computational demands due to its reinforcement learning framework, it consistently outperforms existing algorithms like LEF, ATP and JOTP. The advantages are particularly evident in handling the complexities associated with dynamic task assignment and high item type diversity. These findings highlight the strengths of JOTP-RL in joint optimization of both task assignment and path planning.

(4) **Memory Efficiency:** Despite the more sophisticated approach, JOTP-RL demonstrates superior memory efficiency compared to other advanced algorithms, such as ATP and JOTP. This makes it highly suitable for resource-constrained environments, where real-time task planning is essential. The reduced memory footprint further enhances its practicality in large-scale, real-time warehouse systems.

Overall, these findings emphasize the robust adaptability and superior performance of the JOTP-RL algorithm in complex, real-time scenario. By integrating multi-agent reinforcement learning with batch-based task assignment, path planning efficiency improvements through the THA* algorithm, JOTP-RL stands out as the most effective and practical solution for order picking in intelligent logistics warehousing.

Conclusion and future works

In this paper, we investigate a real-time task planning problem called RTTP for the intelligent logistics warehouse scenario. To address the RTTP problem, the following contributions are presented: (1) First, a joint optimization evaluation indicator is designed, followed by the development of a joint optimization task planning (JOTP) algorithm that identifies the optimal matching between racks and orders within each batch. (2) To further improve task planning effectiveness, the rack selection process is modeled as a partially observable Markov decision process (POMDP). Additionally, a RL-based multi-agent joint optimization algorithm (JOTP-RL) is proposed for task planning, which is solved using the Q-Mix algorithm. (3) Finally, comprehensive experiments on two datasets validate the effectiveness and efficiency of the proposed algorithms in comparison with state-of-the-art methods. These results indicate the effectiveness and efficiency of our JOTP-RL algorithm, providing a solid foundation for future research on real-time task planning in intelligent logistics warehouse.

On the other hand, our study has some limitations: (1) Assumptions about robot speed and power consumption: To simplify our model, we assume that the robot moves at a constant speed and that its electricity consumption increases linearly over time. However, this assumption may underestimate the impact of payload weight on power consumption, which could affect the accuracy of the power consumption model. Future models could incorporate more accurate representations of these dynamics to improve the reliability of power consumption predictions. (2) Limited scope of datasets: In this study, the performance of our proposed algorithms is evaluated on two datasets. While these datasets provide valuable insights, they may not capture the full range of real-world scenarios, such as variations in warehouse layouts, order complexities, or robot heterogeneity. Expanding the experiments to include a broader range of datasets could help assess the generalizability of the algorithms across different operational environments.

Future research on real-time task planning for intelligent logistics warehouses could focus on three main parts: First, Enhancing multi-agent coordination: Improving multi-agent coordination in complex environments by adapting algorithms such as Q-Mix can better address obstacles and disruptions. Second, Optimizing human-robot collaboration: Enhancing human-robot collaboration through dynamic task allocation, considering human factors, can further optimize operational efficiency. Lastly, Integrating advanced perception technologies and scalability: Incorporating advanced perception technologies and exploring scalability will be essential for the practical implementation of larger-scale operations. These advancements are expected to significantly enhance system performance and enable more intelligent warehouse management.

Data availability

The datasets and code generated and/or analyzed during the current study are available from the corresponding author on reasonable request, and the datasets are partially derived from <https://www.kaggle.com/datasets/olisibr/brazilian-ecommerce>.

Received: 20 September 2024; Accepted: 28 January 2025

Published online: 01 March 2025

References

1. Tubis, A. A. & Rohman, J. Intelligent warehouse in industry 4.0—systematic literature review. *Sensors* **23**, 4105. <https://doi.org/10.3390/s23084105> (2023).
2. Kumar, V. V., Chan, F. T. S., Mishra, N. & Kumar, V. Environmental integrated closed loop logistics model: An artificial bee colony approach. In *8th International Conference on Supply Chain Management and Information Systems (SCMIS)*. 1–7. <https://ieeexplore.ieee.org/document/5681670> (2010).

3. Kumar, V. V. & Chan, F. T. S. A superiority search and optimization algorithm to solve RFID and an environmental factor embedded closed loop logistics model. *Int. J. Prod. Res.* **49**, 4807–4831. <https://doi.org/10.1080/00207543.2010.503201> (2011).
4. Kumar, V. V., Liou, F. W., Balakrishnan, S. N. & Kumar, V. Economical impact of RFID implementation in remanufacturing: a chaos-based interactive artificial bee colony approach. *J. Intell. Manuf.* **26**, 815–830. <https://doi.org/10.1007/s10845-013-0836-9> (2015).
5. Barreto, L., Amaral, A. & Pereira, T. Industry 4.0 implications in logistics: an overview. *Proc. Manuf.* **13**, 1245–1252. <https://doi.org/10.1016/j.promfg.2017.09.045> (2017).
6. Kumar, V. V., Sahoo, A. & Liou, F. W. Cyber-enabled product lifecycle management: A multi-agent framework. *Proc. Manuf.* **39**, 123–131. <https://doi.org/10.1016/j.promfg.2020.01.247> (2019).
7. Huang, G. Q., Chen, M. Z. & Pan, J. Robotics in ecommerce logistics. *HKIE Trans.* **22**, 68–77. <https://doi.org/10.1080/1023697X.2015.1043960> (2015).
8. Huang, T. et al. Benchmarking quantum (-inspired) annealing hardware on practical use cases. *IEEE Trans. Comput.* **72**, 1692–1705. <https://doi.org/10.1109/TC.2022.3219257> (2022).
9. Li, Y. & Huang, H. Efficient task planning for heterogeneous AGVs in warehouses. *IEEE Trans. Intell. Transp. Syst.* **25**, 10005–10019. <https://doi.org/10.1109/TITS.2024.3356514> (2024).
10. Mao, J., Cheng, J., Li, X. & Cao, B. Research on scheduling optimization of four-way shuttle-based storage and retrieval systems. *Sci. Rep.* **13**, 3999. <https://doi.org/10.1038/s41598-023-31050-8> (2023).
11. Shi, D. et al. Adaptive task planning for large-scale robotized warehouses. In *IEEE 38th International Conference on Data Engineering (ICDE)*, 3327–3339. <https://doi.org/10.1109/ICDE53745.2022.00314> (2022).
12. Shi, D. et al. Collision-aware route planning in warehouses made efficient: A strip-based framework. In *IEEE International Conference on Data Engineering (ICDE)*, 869–881. <https://doi.org/10.1109/ICDE55515.2023.00072> (2023).
13. Yang, X., Hua, G., Hu, L., Cheng, T. C. E. & Huang, A. Joint optimization of order sequencing and rack scheduling in the robotic mobile fulfilment system. *Comput. Oper. Res.* **135**, 105467. <https://doi.org/10.1016/j.COR.2021.105467> (2021).
14. Zhuang, Y., Zhou, Y., Yuan, Y., Hu, X. & Hassini, E. Order picking optimization with rack-moving mobile robots and multiple workstations. *Eur. J. Oper. Res.* **300**, 527–544. <https://doi.org/10.1016/j.EJOR.2021.08.003> (2022).
15. Sharon, G., Stern, R., Felner, A. & Sturtevant, N. R. Conflict-based search for optimal multi-agent pathfinding. *Artif. Intell.* **219**, 40–66. <https://doi.org/10.1016/j.ARTINT.2014.11.006> (2015).
16. Barer, M., Sharon, G., Stern, R. & Felner, A. Suboptimal variants of the conflict-based search algorithm for the multi-agent pathfinding problem. *Front. Artif. Intell. Appl.* **263**, 961–962. <https://doi.org/10.3233/978-1-61499-419-0-961> (2014).
17. Surynek, P., Felner, A., Stern, R. & Boyarski, E. Sub-optimal sat-based approach to multi-agent path-finding problem. In *Proceedings of the International Symposium on Combinatorial Search*, 9, 99–105. <https://doi.org/10.1609/SOCS.V9I1.18456> (2018).
18. Valle, C. A. & Beasley, J. E. Order allocation, rack allocation and rack sequencing for pickers in a mobile rack environment. *Comput. Oper. Res.* **125**, 105090. <https://doi.org/10.1016/j.COR.2020.105090> (2021).
19. Xie, L., Thieme, N., Krenzler, R. & Li, H. Introducing split orders and optimizing operational policies in robotic mobile fulfillment systems. *Eur. J. Oper. Res.* **288**, 80–97. <https://doi.org/10.1016/j.EJOR.2020.05.032> (2021).
20. Chou, S., Mauliddina, A. N., Dewabharata, A. & Zulvia, F. E. Pick order assignment and order batching strategy for robotic mobile fulfillment system warehouse. In *Winter Simulation Conference (WSC)*, 1700–1711. <https://doi.org/10.1109/WSC60868.2023.10408409> (2023).
21. Li, Y., Côté, J., Coelho, L. C., Zhang, C. & Zhang, S. Order assignment and scheduling under processing and distribution time uncertainty. *Eur. J. Oper. Res.* **305**, 148–163. <https://doi.org/10.1016/j.EJOR.2022.05.033> (2023).
22. Lin, S., Liu, A., Wang, J. & Kong, X. An intelligence-based hybrid pso-sa for mobile robot path planning in warehouse. *J. Comput. Sci.* **67**, 101938. <https://doi.org/10.1016/j.JOCS.2022.101938> (2023).
23. Maoudj, A. & Christensen, A. L. Decentralized multi-agent path finding in dynamic warehouse environments. In *2023 21st International Conference on Advanced Robotics (ICAR)*, 28–34. <https://doi.org/10.1109/ICAR58858.2023.10406648> (2023).
24. Kanai, M., Ishihara, S., Narikawa, R. & Ohtsuka, T. Efficient path planning of warehouse robots utilizing model predictive control. In *IEEE 28th International Conference on Emerging Technologies and Factory Automation (ETFA)*, 1–8. <https://doi.org/10.1109/ETFA54631.2023.10275638> (2023).
25. Tang, L., Yang, C., Wen, K., Wu, W. & Guo, Y. Quantum computing for several agv scheduling models. *Sci. Rep.* **14**, 12205. <https://doi.org/10.1038/s41598-024-62821-6> (2024).
26. Li, Y. et al. Utility-aware dynamic ridesharing in spatial crowdsourcing. *IEEE Trans. Mobile Comput. (TMC)* **23**, 1066–1079. <https://doi.org/10.1109/TMC.2022.3232215> (2022).
27. Tong, Y. et al. Flexible online task assignment in real-time spatial data. *Proc. VLDB Endowment* **10**, 1334–1345. <https://doi.org/10.14778/3137628.3137643> (2017).
28. Zhao, Y. et al. Predictive task assignment in spatial crowdsourcing: A data-driven approach. In *IEEE 36th International Conference on Data Engineering (ICDE)*, 13–24. <https://doi.org/10.1109/ICDE48307.2020.00009> (2020).
29. Tong, Y. et al. Combinatorial optimization meets reinforcement learning: Effective taxi order dispatching at large-scale. *IEEE Trans. Knowl. Data Eng. (TKDE)* **35**, 9812–9823. <https://doi.org/10.1109/TKDE.2021.3127077> (2021).
30. Wang, L. et al. Space-ta: Cost-effective task allocation exploiting intradata and interdata correlations in sparse crowdsensing. *ACM Trans. Intell. Syst. Technol. (TIST)* **9**, 1–28. <https://doi.org/10.1145/3131671> (2017).
31. Wurman, P. R., D'Andrea, R. & Mountz, M. Coordinating hundreds of cooperative, autonomous vehicles in warehouses. *AI Mag.* **9**. <https://doi.org/10.1609/aimag.v29i1.2082> (2008).
32. Tong, Y., Zeng, Y., Zhou, Z., Chen, L. & Xu, K. Unified route planning for shared mobility: An insertion-based framework. *ACM Trans. Database Syst. (TODS)* **47**, 1–48. <https://doi.org/10.1145/3488723> (2022).
33. Luo, Z. et al. Diversified top-k route planning in road network. *Proc. VLDB Endow.* **15**, 3199–3212. <https://doi.org/10.14778/3551793.3551863> (2022).
34. Zhang, Q., Wu, K. & Shi, Y. Route planning and power management for phev with reinforcement learning. *IEEE Trans. Veh. Technol.* **69**, 4751–4762. <https://doi.org/10.1109/TVT.2020.2979623> (2020).
35. Sartoretti, G. et al. Primal: Pathfinding via reinforcement and imitation multi-agent learning. *IEEE Robot. Autom. Lett.* **4**, 2378–2385. <https://doi.org/10.1109/LRA.2019.2903261> (2019).
36. Liu, Z. et al. Mapper: Multi-agent path planning with evolutionary reinforcement learning in mixed dynamic environments. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 11748–11754. <https://doi.org/10.1109/IROS45743.2020.9340876> (2020).
37. Kuhn, H. W. The hungarian method for the assignment problem. *Nav. Res. Logist.* **2**, 83–97. https://doi.org/10.1007/978-3-540-68279-0_2 (1955).
38. Munkres, J. Algorithms for the assignment and transportation problems. *J. Soc. Ind. Appl. Math.* **5**, 32–38. <https://doi.org/10.1137/0105003> (1957).
39. Rashid, T. et al. QMIX: monotonic value function factorisation for deep multi-agent reinforcement learning. In *Proceedings of the 35th International Conference on Machine Learning*, vol. 80, 4292–4301. <https://doi.org/10.48550/arXiv.1803.11485> (2018).
40. Lowe, R. et al. Multi-agent actor-critic for mixed cooperative-competitive environments. In *Advances in Neural Information Processing Systems*, 6379–6390. <https://doi.org/10.48550/arXiv.1706.02275> (2017).

41. Ikeda, T. & Shibuya, T. Centralized training with decentralized execution reinforcement learning for cooperative multi-agent systems with communication delay. In *61st IEEE Annual Conference of the Society of Instrument and Control Engineers*. 135–140. <https://doi.org/10.23919/SICE56594.2022.9905866> (2022).
42. Rashid, T. et al. Monotonic value function factorisation for deep multi-agent reinforcement learning. *J. Mach. Learn. Res.* **21**, 1–51. <https://doi.org/10.48550/arXiv.2104.01939> (2020).
43. Hart, P. E., Nilsson, N. J. & Raphael, B. A formal basis for the heuristic determination of minimum cost paths. *IEEE Trans. Syst. Sci. Cybern.* **4**, 100–107. <https://doi.org/10.1109/TSSC.1968.300136> (1968).
44. Deng, D., Shahabi, C., Demiryurek, U. & Zhu, L. Task selection in spatial crowdsourcing from worker’s perspective. *GeoInformatica* **20**, 529–568. <https://doi.org/10.1007/s10707-016-0251-4> (2016).
45. Boysen, N., Briskorn, D. & Emde, S. Parts-to-picker based order processing in a rack-moving mobile robots environment. *Eur. J. Oper. Res.* **262**, 550–562. <https://doi.org/10.1016/j.ejor.2017.03.053> (2017).
46. Dang, Q. V., Singh, N., Adan, I., Martagan, T. & van de Sande, D. Scheduling heterogeneous multi-load AGVs with battery constraints. *Comput. Oper. Res.* **136**, 105517. <https://doi.org/10.1016/j.cor.2021.105517> (2021).

Acknowledgements

Here, we are deeply grateful to all reviewers and editors. This work was supported by the National Natural Science Foundation of China (No. 62172457), Science and Technology Research Project in Henan Province of China (No. 242102210104, 252102210032), the Key Scientific Research Project in Henan University of China (No. 25B520021).

Author contributions

S.Z. contributed to the conceptualization, study design, data collection, interpretation, and manuscript writing. Q.H. contributed to data collection, literature search, analysis, and interpretation. H.Z. and H.W. contributed to conceptualization, analysis, supervision and fund sourcing. H.L. and K.W. contributed to the data analysis, interpretation, writing of the manuscript, and experiment validation. All authors discussed the results and contributed to the final manuscript.

Declarations

Competing interests

The authors declare no competing interests.

Additional information

Correspondence and requests for materials should be addressed to S.Z. or H.Z.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher’s note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article’s Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article’s Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

© The Author(s) 2025